CHAPTER 9

Nondeterministic Finite Automate

Nondeterminism

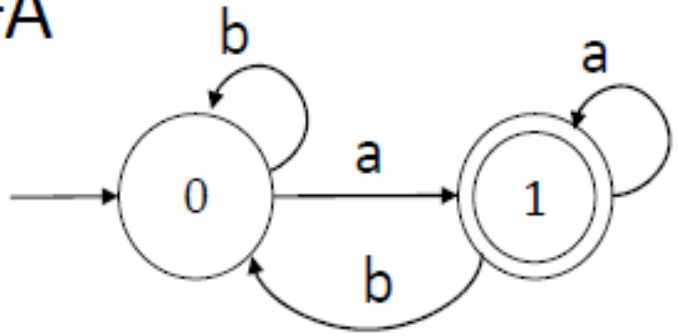
- In a nondeterministic machine, several choices may exist for the next state at any point.
- Nondeterministic Finite Automata (NFA)
- Nondeterminism is a generalization of the determinism, so every DFA is automatically a NFA.

Differences between DFA & NFA

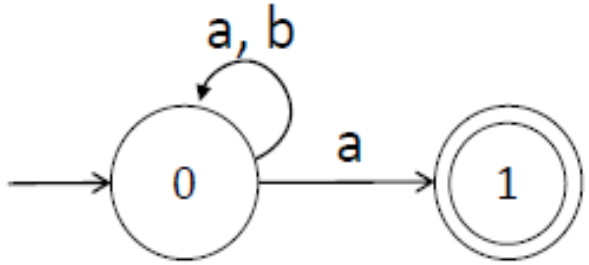
- Every state of DFA always has exactly one exiting transition arrow for each symbol in the alphabet while the NFA may has zero, one or more.
- In a DFA, labels on the transition arrows are from the alphabet while NFA can have an arrow with the label ε.

Example of DFA & NFA

- L={w {a,b}:w ends with a}
  - DFA



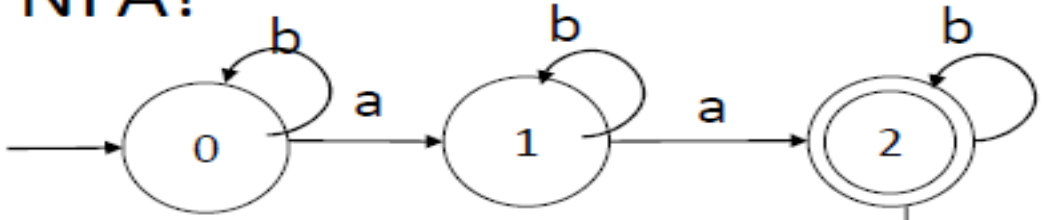
  - NFA



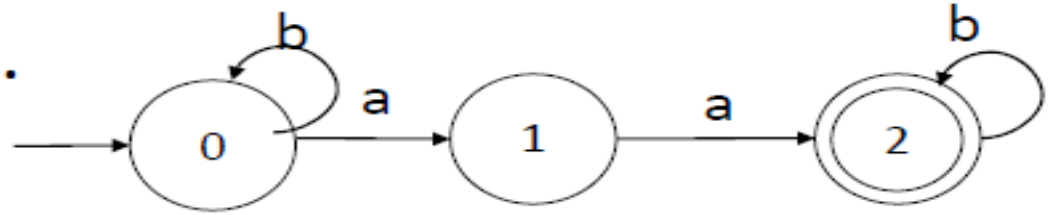
EXERCISE

- DFA or NFA?

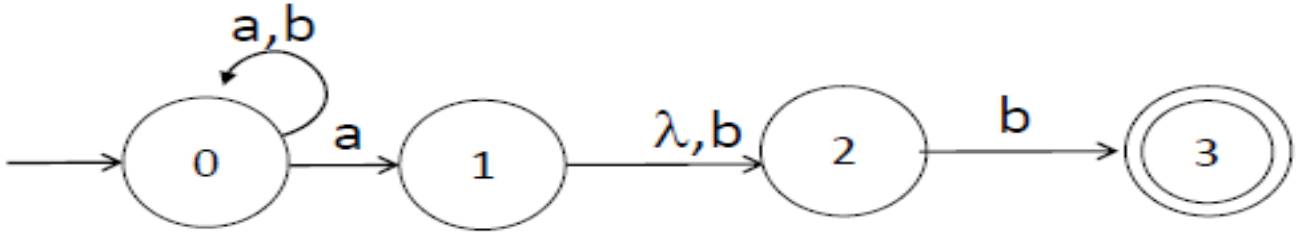
1.



2.

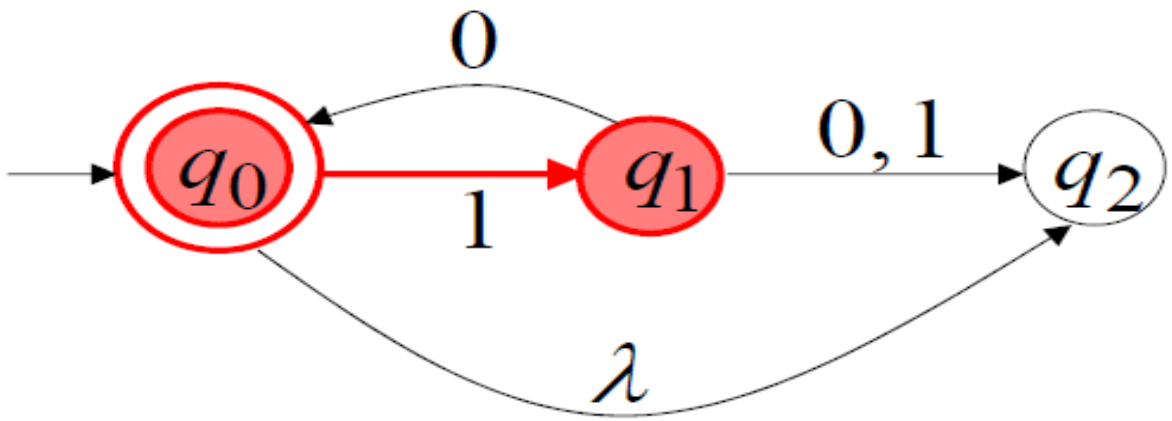


3.



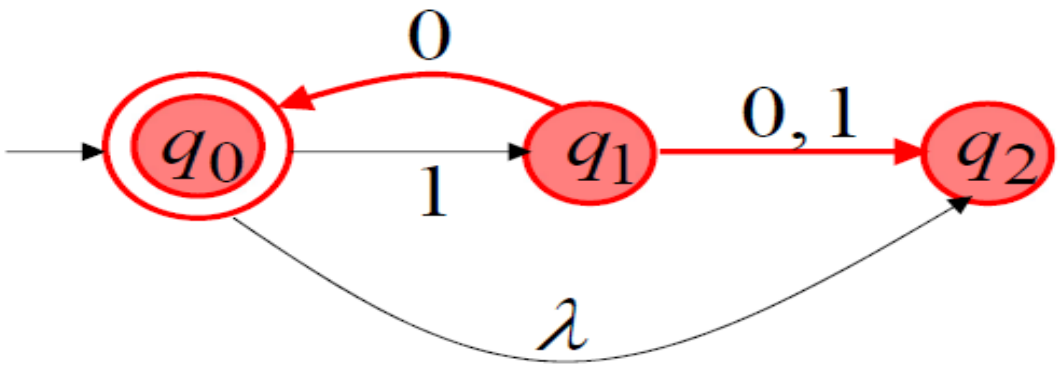
One exiting transition arrow

$$\delta(q_0, 1) = \{q_1\}$$



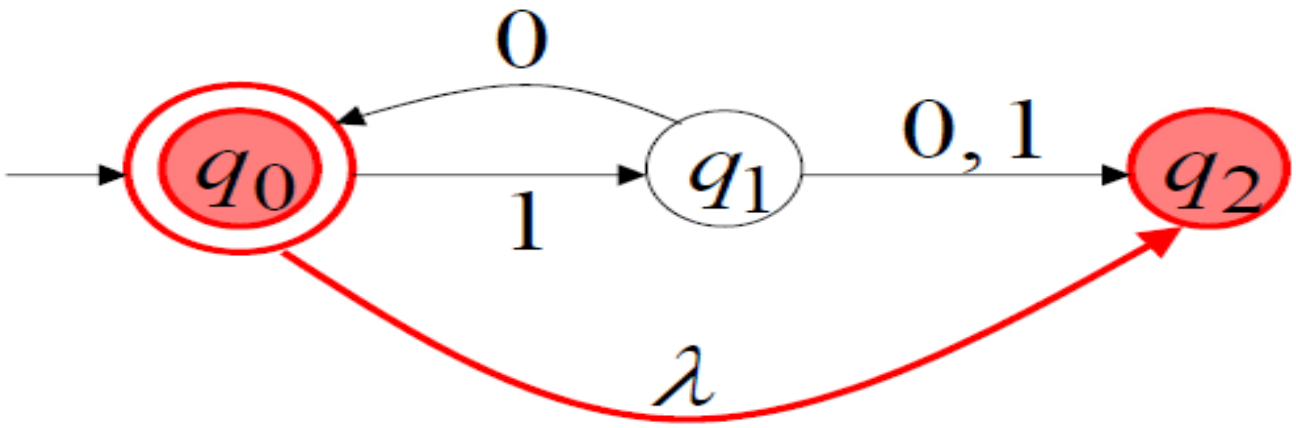
More (two) exiting transition arrows

$$\delta(q_1, 0) = \{q_0, q_2\}$$



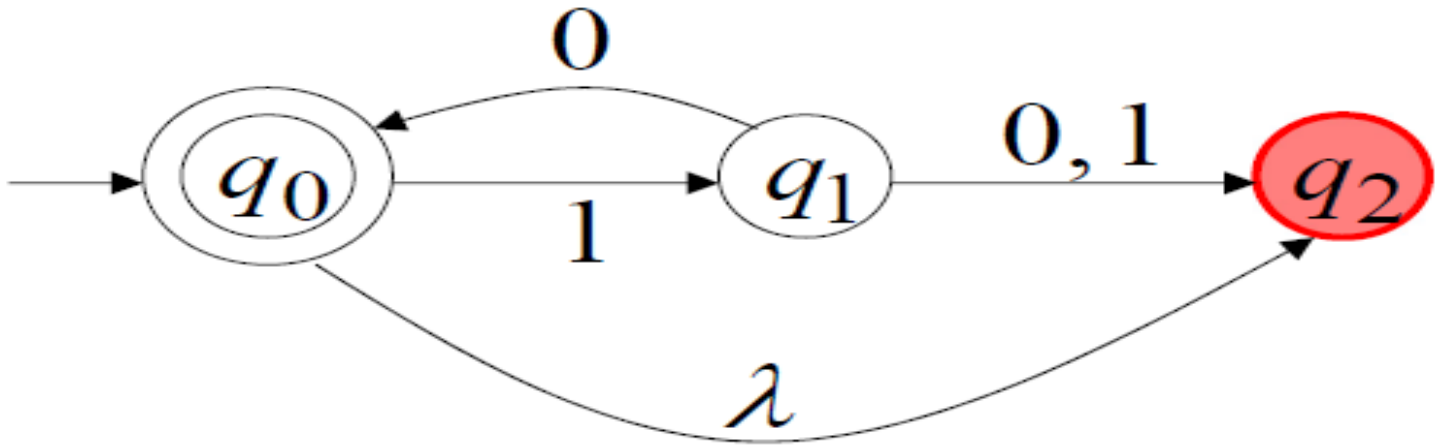
More (two) exiting transition arrows

$$\delta(q_0, \lambda) = \{q_0, q_2\}$$



Zero exiting transition arrow undefined transition

$$\delta(q_2, 1) = \varnothing$$



DFA VS NFA

Features	DFA	NFA
Choice of transition	Single	Multiple
Input label	Alphabets only	Alphabets of λ or ε
Undefined transition	No	Possible

Formal definition of NFA

- A NFA is a 5-tuple ($Q, \sum, \delta, q_0, F$) , where
- – Q is a finite set of states,
- – $\sum$ is a finite alphabet,
- – δ : Q x $\sum\varepsilon \rightarrow$ P(Q) is the transition function,
- – q_0 is the start state, and
- – $F \subseteq Q$ is the set of accept states.

Notation:

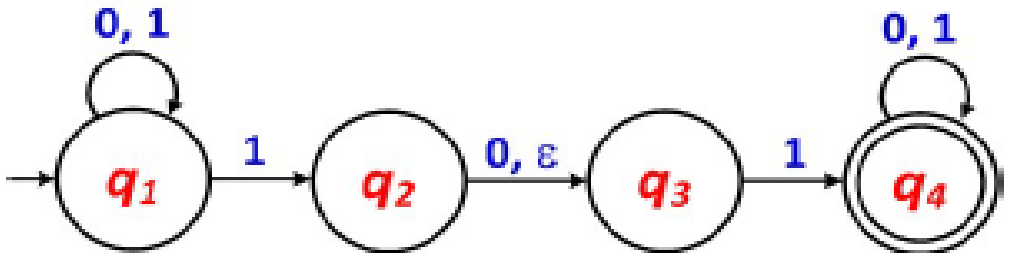
P(Q) is called power set of Q (a collection of all subsets of Q).

and $\sum\varepsilon = \sum \cup \{\varepsilon\}$

Example

- Formal definition of NFA

NFA M_1:



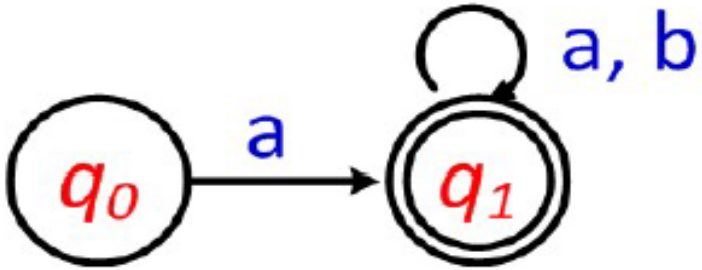
- $M_1: (Q, \Sigma, \delta, q_0, F)$
- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma\{0,1\}$
- q_0 : initial state
- F:$\{q_4\}$

δ

	0	1	ε
q_1	$\{q_1\}$	$\{q_1,q_2\}$	∅
q_2	$\{q_3\}$	∅	$\{q_3\}$
q_3	∅	$\{q_4\}$	∅
q_4	$\{q_4\}$	$\{q_4\}$	∅

Example NFA

- L = {w {a, b} : w starts with a}
- What is the RE for the language?



- What happen if the 1st input is b?
  - Machine “crashes” or hang

How does the NFA work?

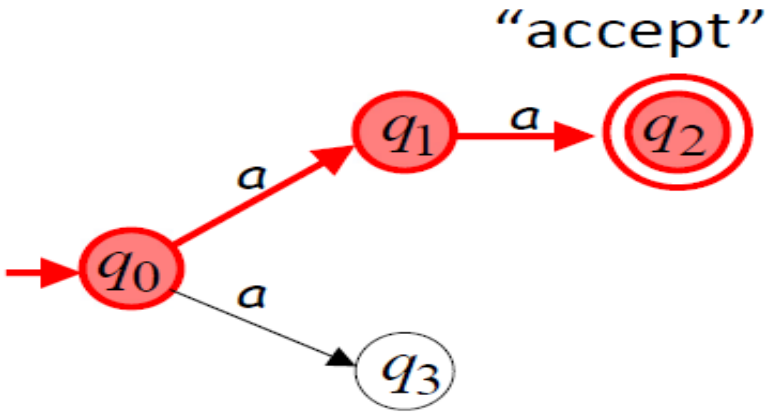
- At a state with multiple transitions (including ε), the NFA splits into **copies** to explore all possibilities simultaneously.
- Each copy follows one transition path. If more choices arise, it splits again.
- A copy **terminates** if no valid transition matches the next input symbol.
- The NFA **accepts** the input if at least one copy reaches an accept state by the end of the input.

Accept or reject string

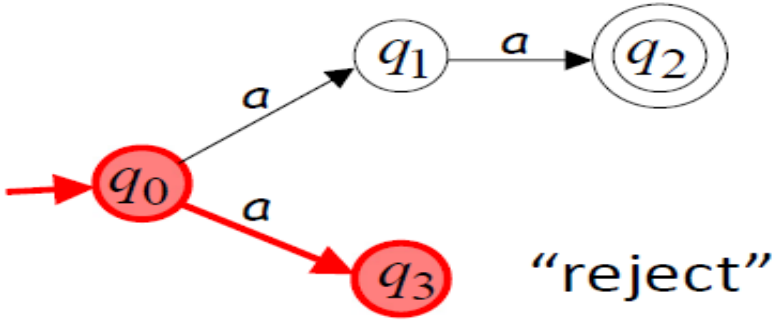
- NFA accepts a string:
  - If at least one computation consumes the entire input and ends in an accepting state.
- NFA rejects a string:
  - If no computation leads to an accepting state after consuming the input
  - The input is fully consumed, but the automaton is in a non-final state.
  - The input cannot be consumed

Example

- *aa* is accepted by NFA

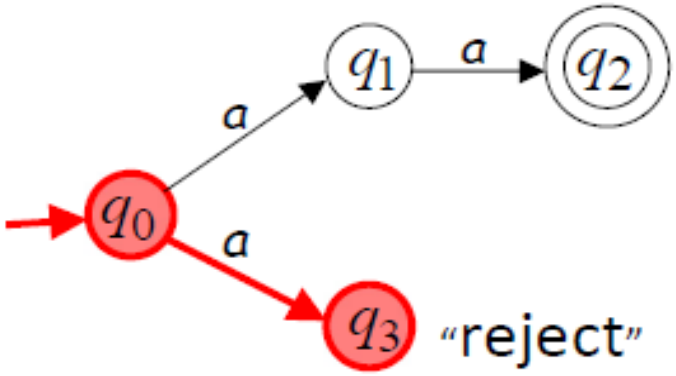


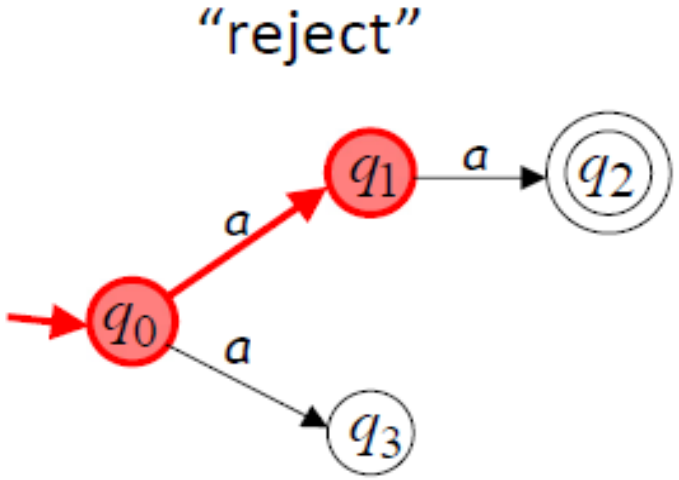
because this computation accepts aa



Example

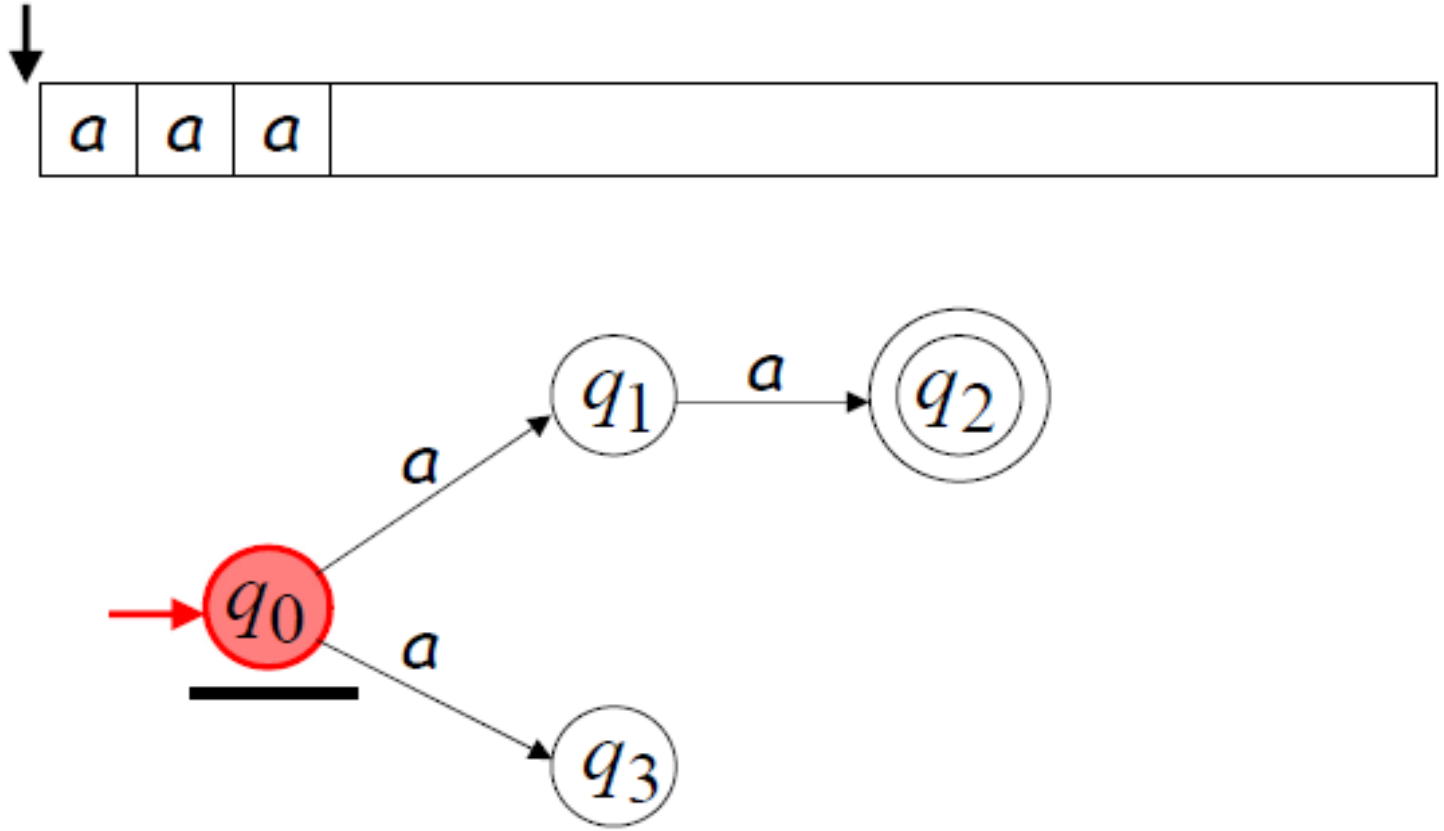
- *a* is rejected by the NFA





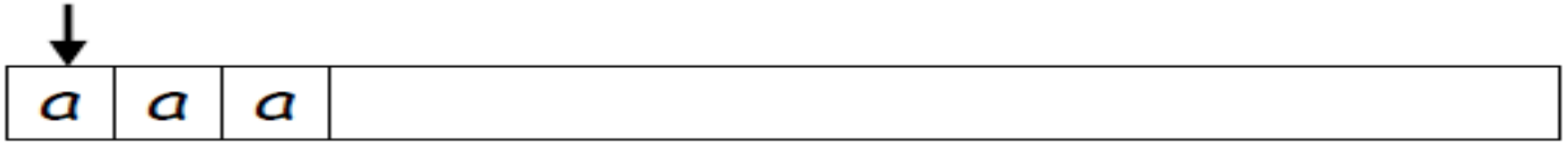
- All possible computations lead to rejection

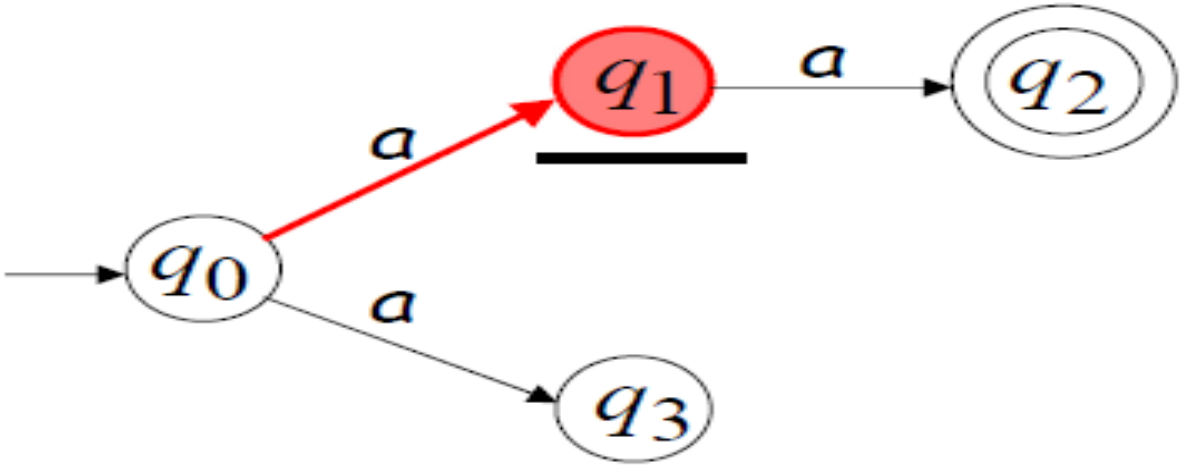
Rejection Example



Rejection Example

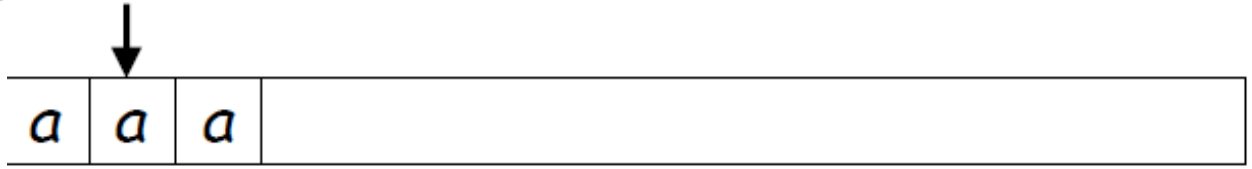
First Choice

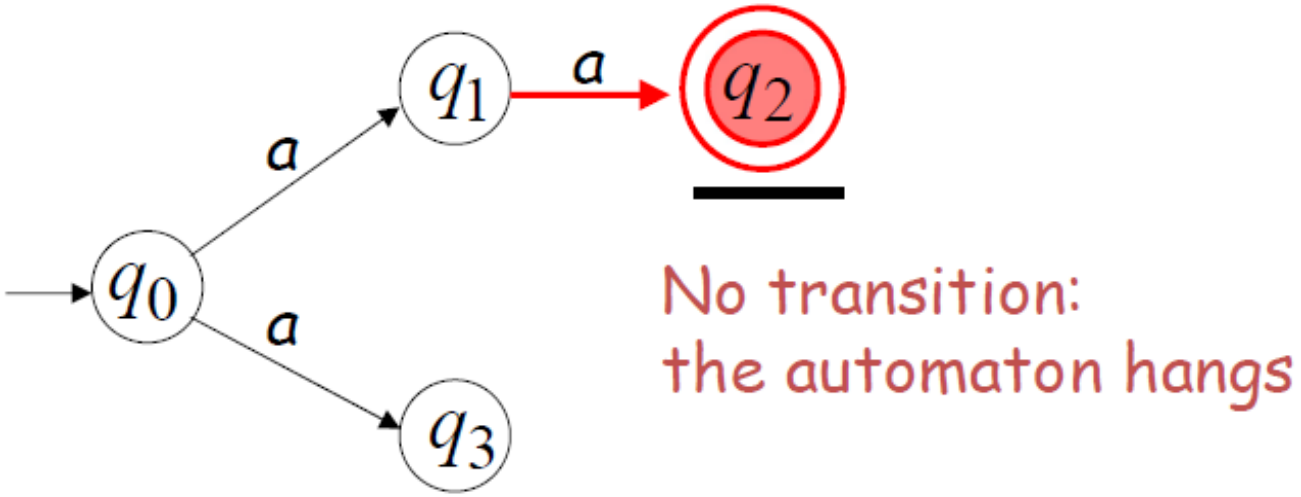




Rejection Example

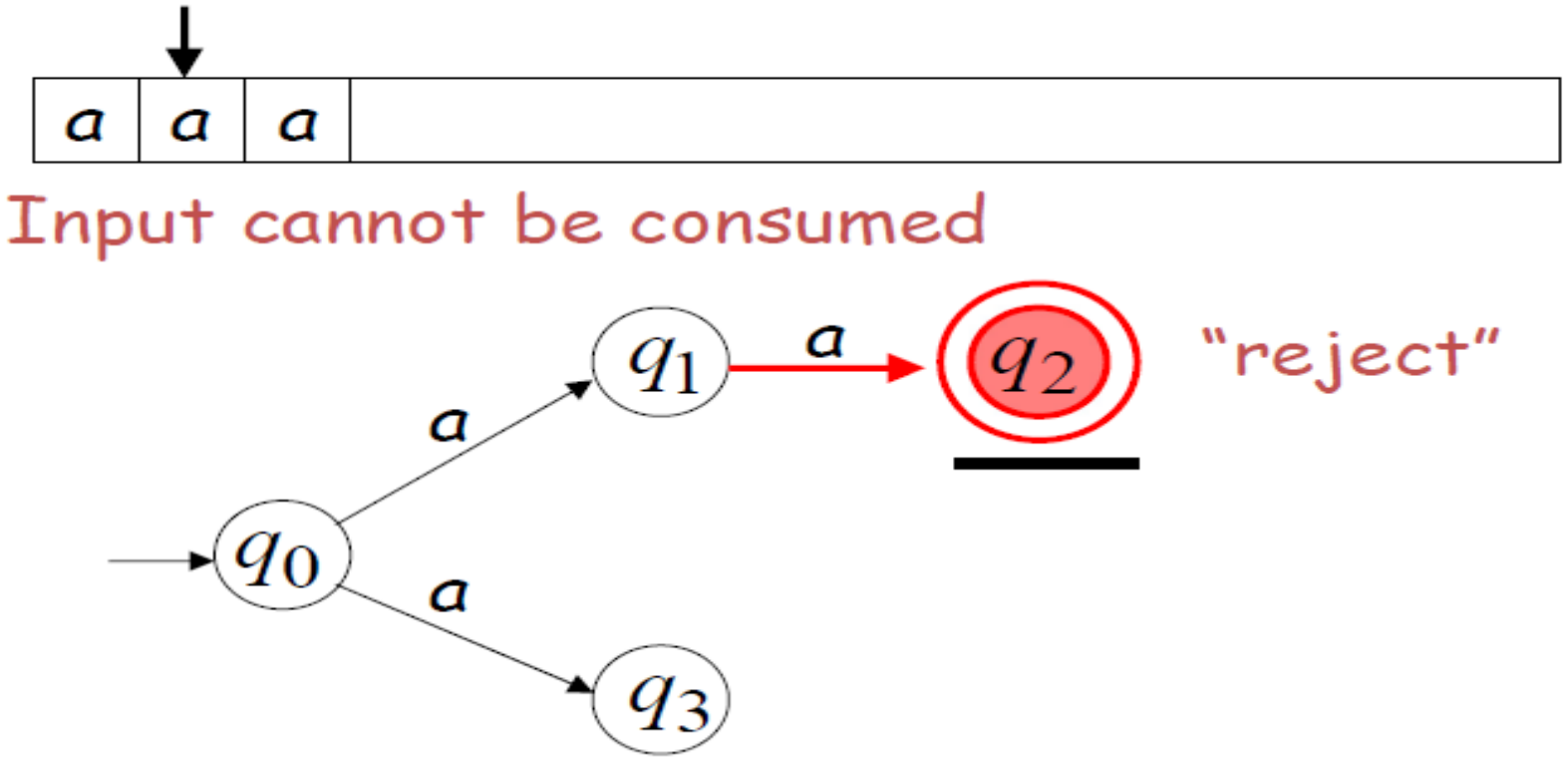
- First choice





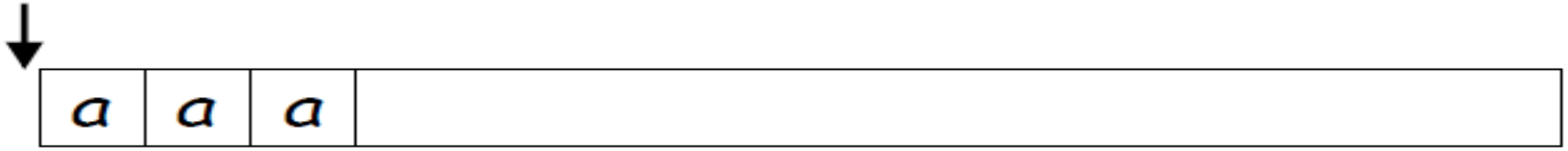
Rejection Example

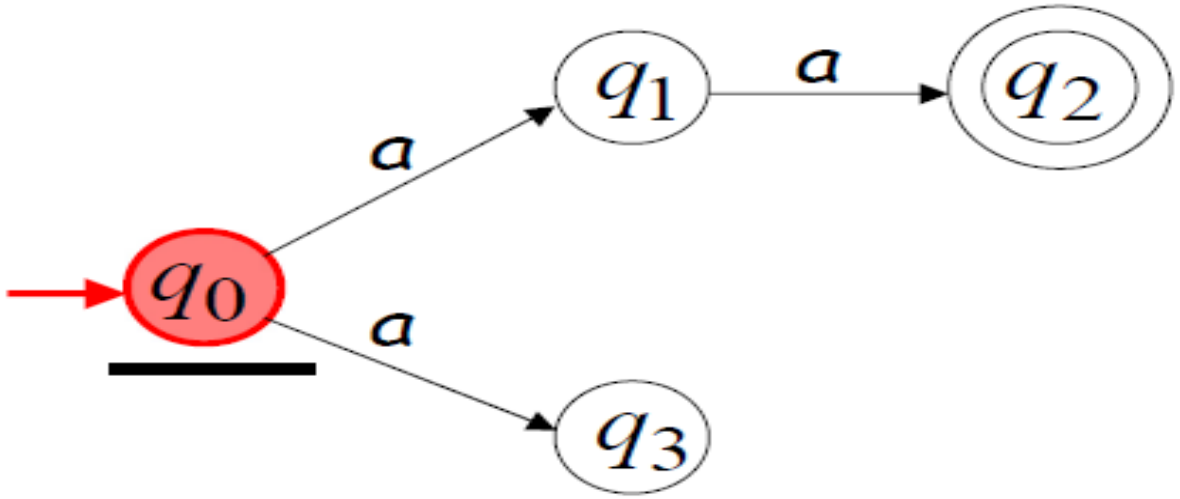
- First choice



Rejection Example

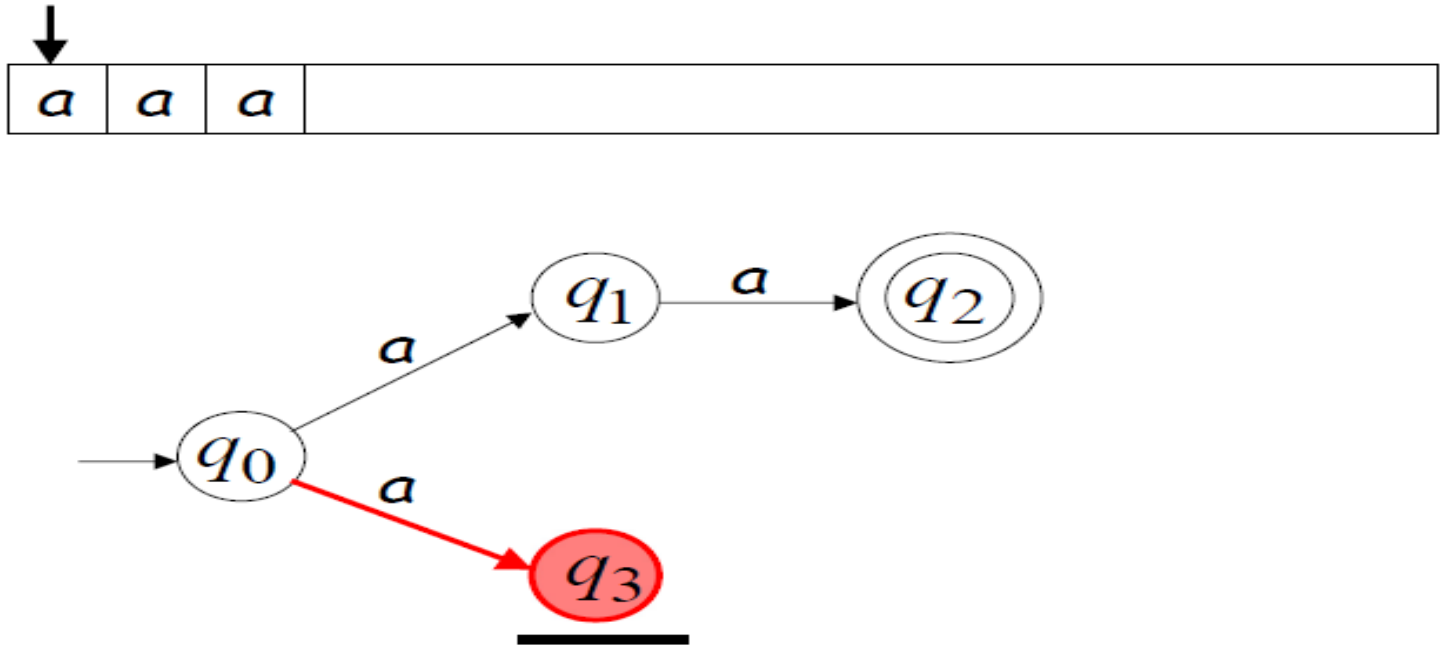
- Second Choice





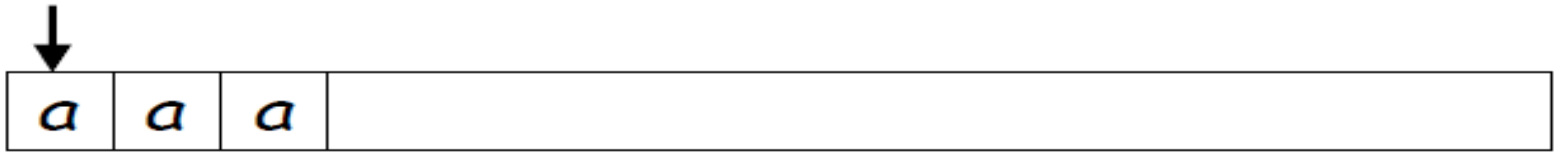
Rejection Example

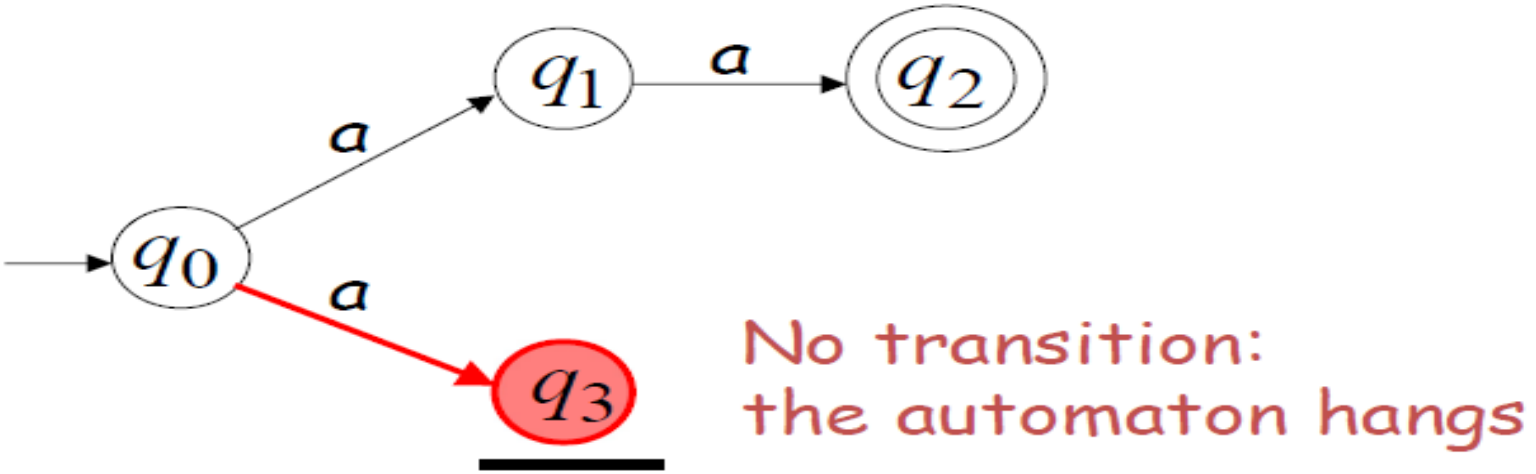
- Second choice



Rejection Example

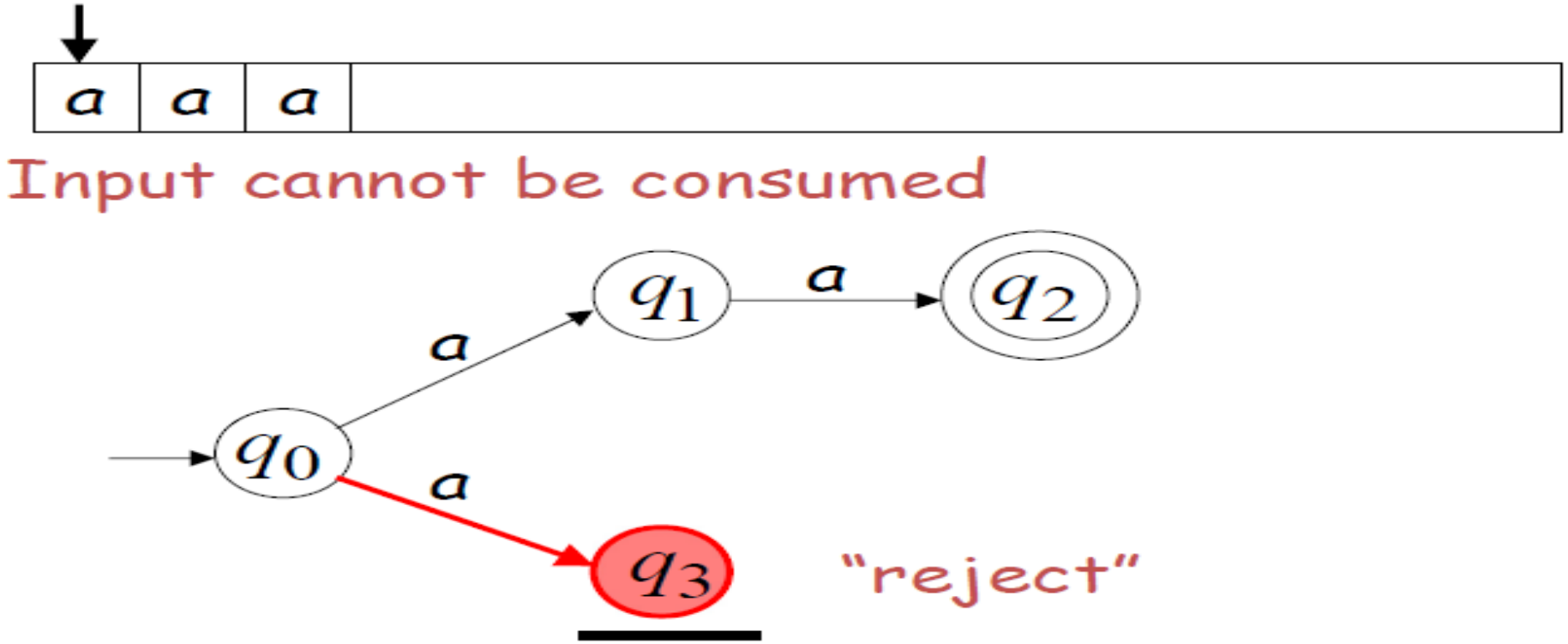
- Second choice





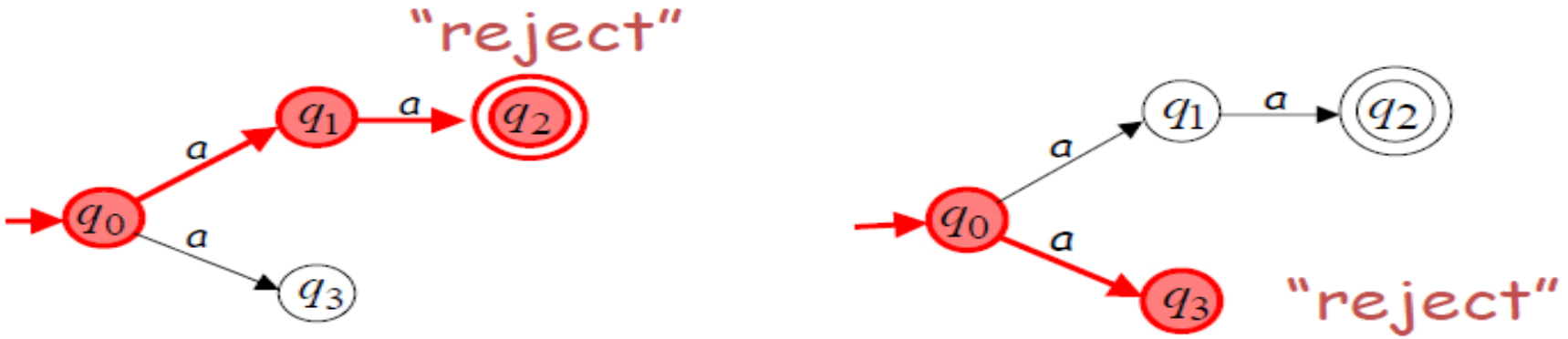
Rejection Example

- Second choice



Example

- *aaa* is rejected by NFA

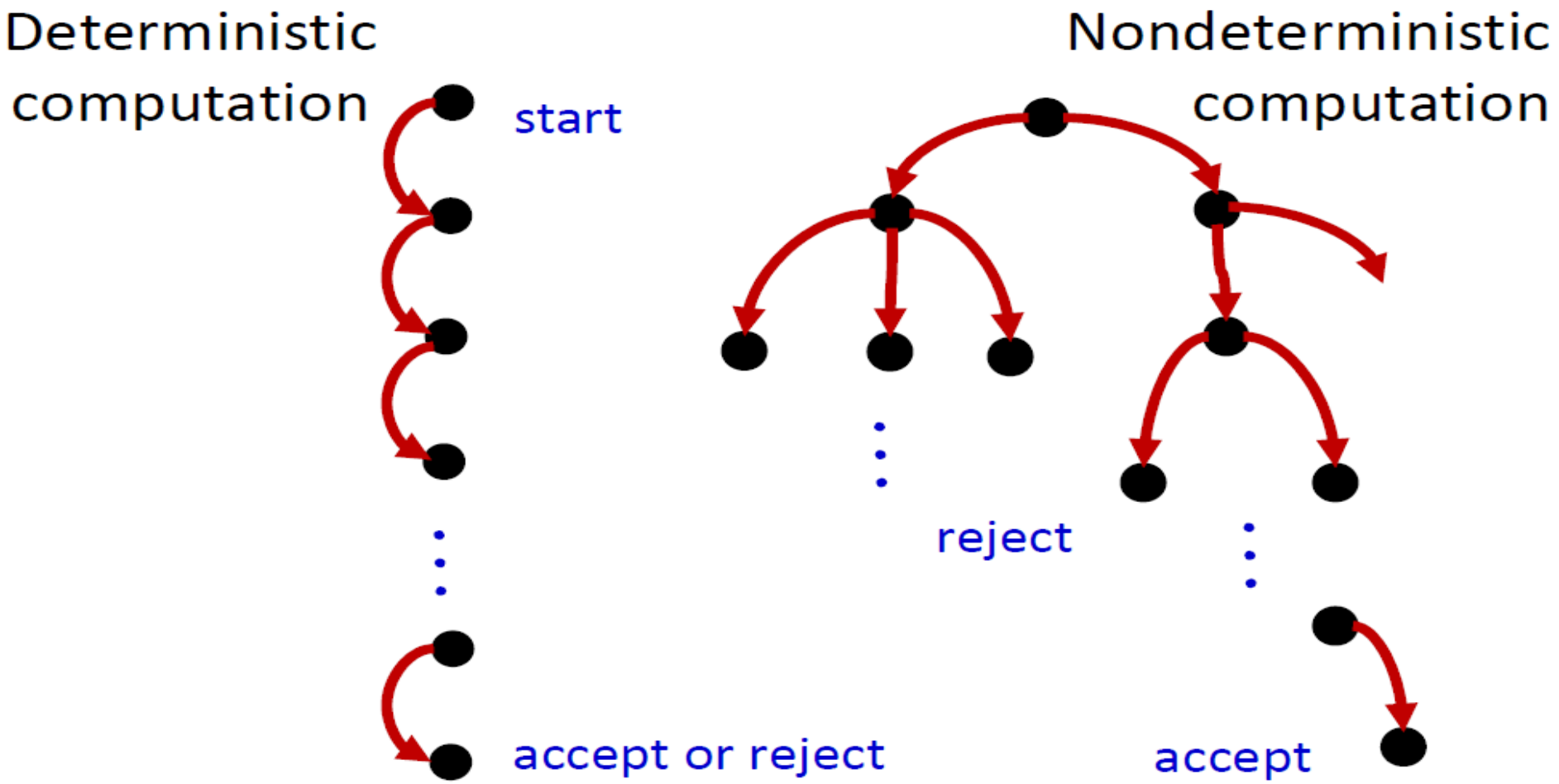


- All possible computations lead to rejection

Tree of possibilities

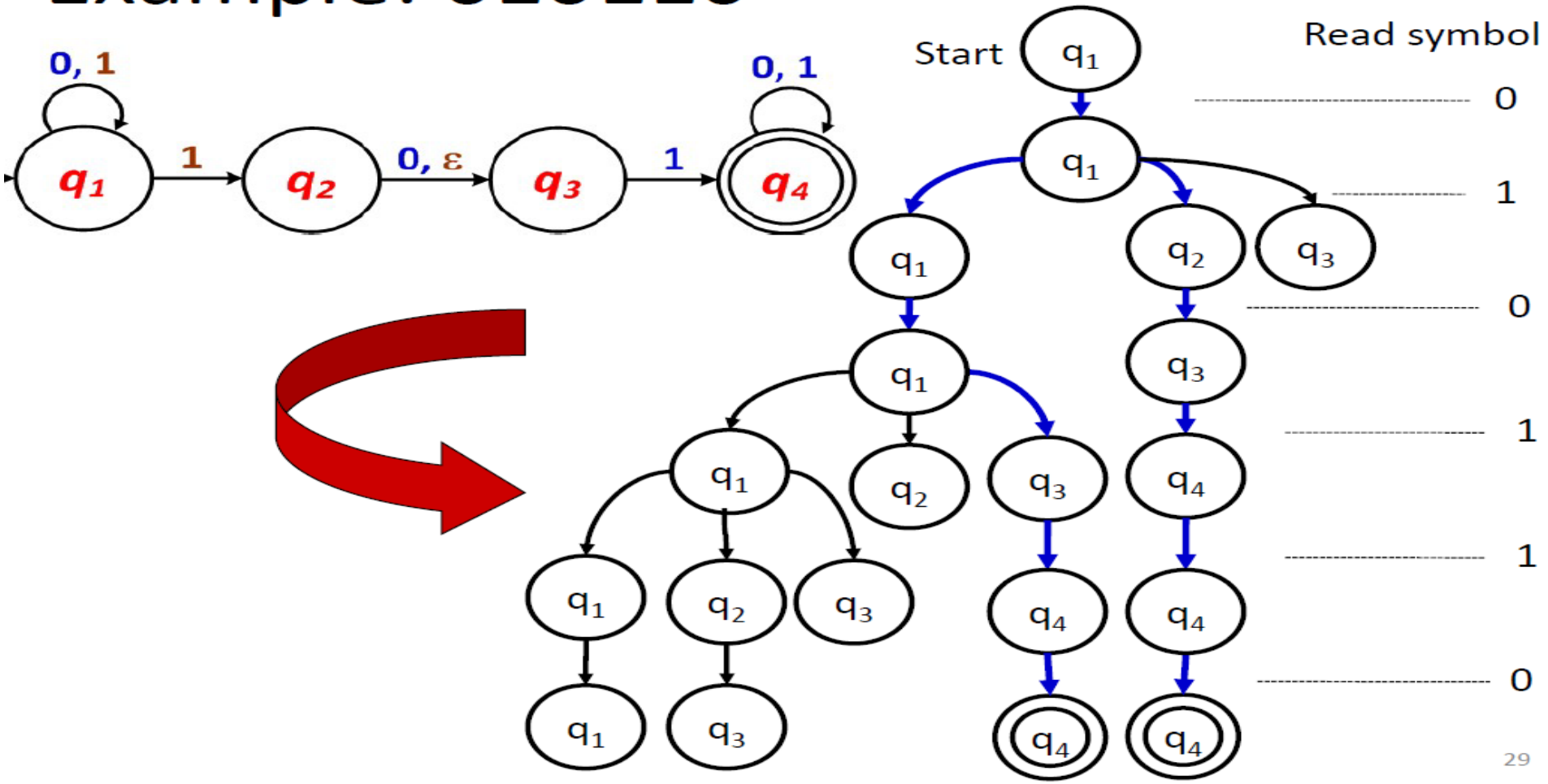
- Think of a nondeterministic computation as a tree of possibilities
- The root of the tree corresponds to the start of the computation.
- Every branch point in the tree corresponds to a point in the computation at which the machine has multiple choices.
- The machine accepts if at least one of the computation branches ends in the an accept state.

Tree of possibilities

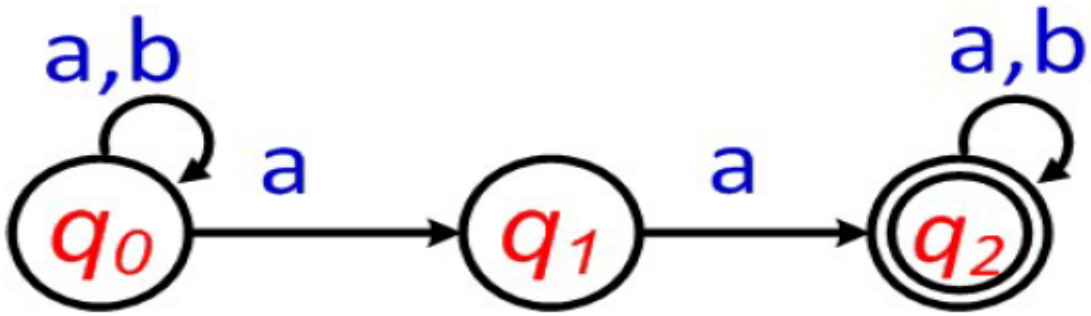


Example

Example: 010110



Example of NFA

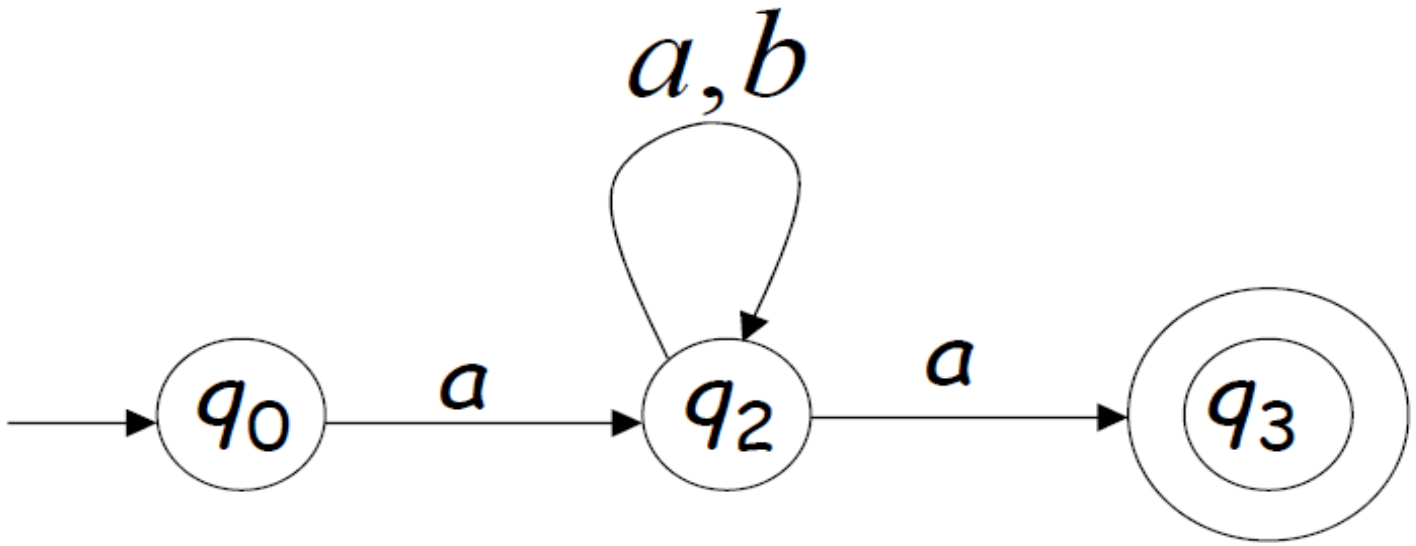


- L = {w ∈ {a, b} : w contains the substring aa}
- What is the RE for the language?
- What happen if the 1st input is *a*?
- Does the machine accept *abaa*?

(q0, abaa) → (q0, baa) → (q0,aa) → (q0, a) → (q0, e) reject
(q0, abaa) → (q0, baa) → (q0,aa) → (q0, a) → (q1, e) reject
(q0, abaa) → (q0, baa) → (q0,aa) → (q1, a) → (q2, e) accept
(q0, abaa) → (q1, baa) reject

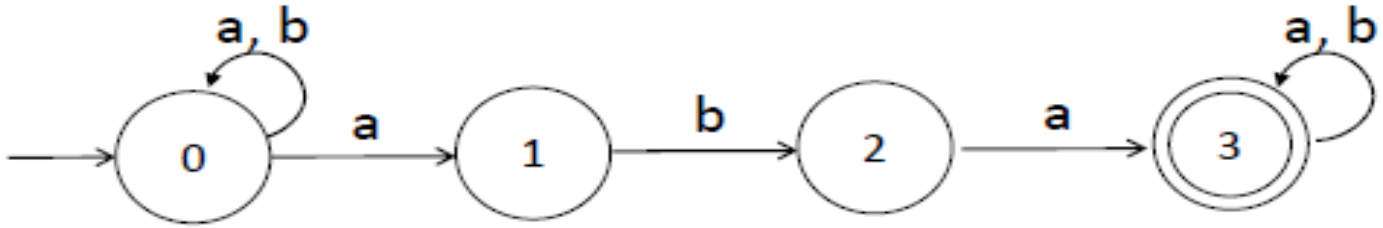
Example (NFA or DFA?)

- Create a FA for $L(M) = \{awa : w \in \{a, b\}^*\}$



- What happen if the 1st input is *b*?
- What happen if the input are *bab*, *abab*, … ?

Exercise



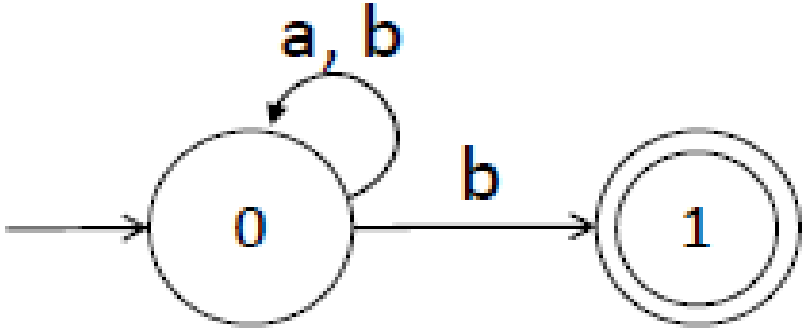
a) Input aabaa, accept or reject?

b) Input baabb, accept or reject?

Design NFA

- L1 = {end with b} Σ={a, b}

RE : (a+b)*b



L2={start with ab} Σ={a, b}

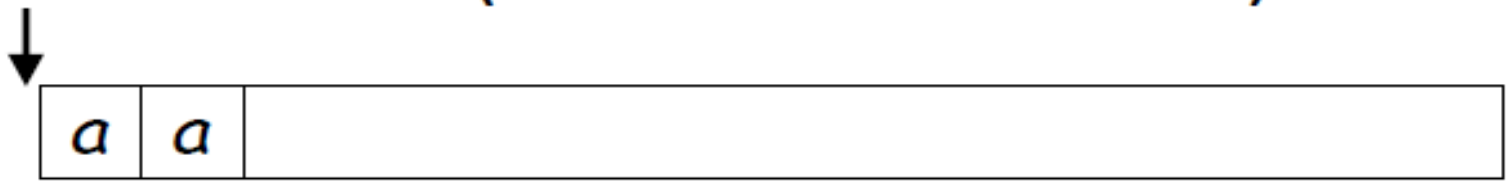
Design NFA

- L3 = {contain ‘aa’}
- L4 = {string of length 2} Σ={a, b}
- L5={string of length at most 3}

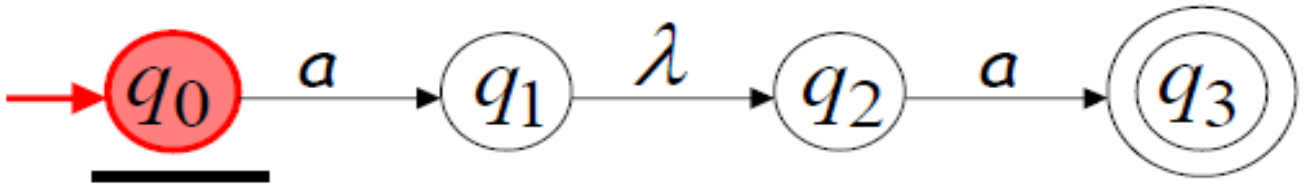
NFA (with λ-transition)

- λ-transition consumes no input
- The λ symbol never appears on the input tape
- It can do a state transition without reading input

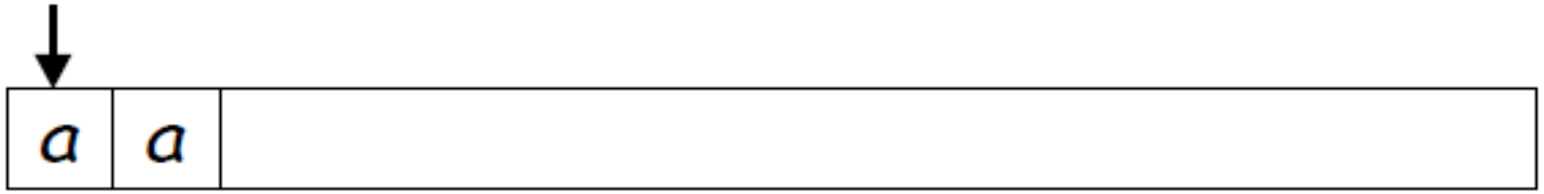
NFA (with λ-transition)

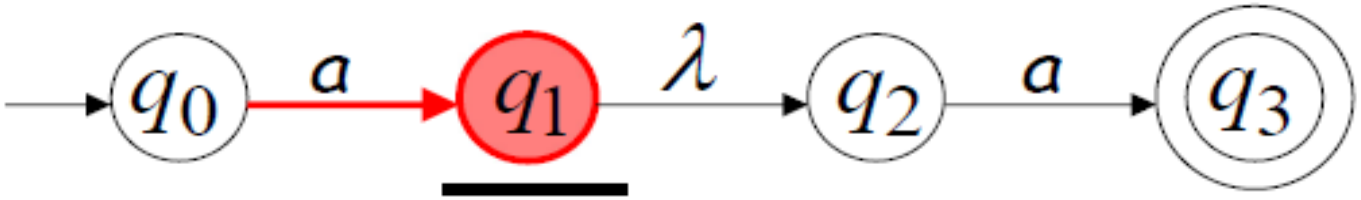


- Input string aa

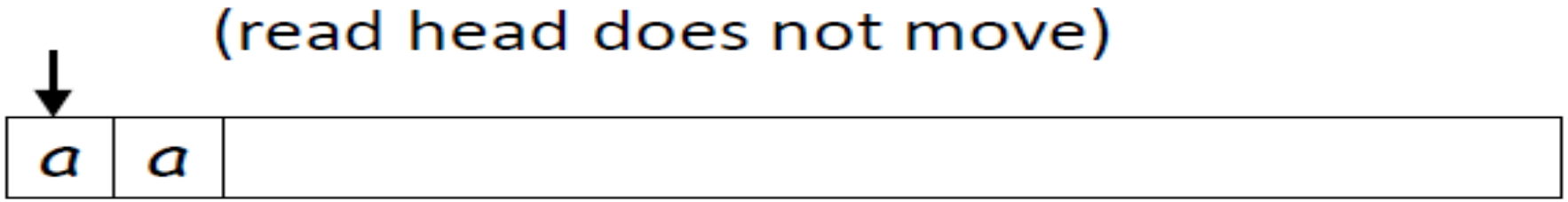


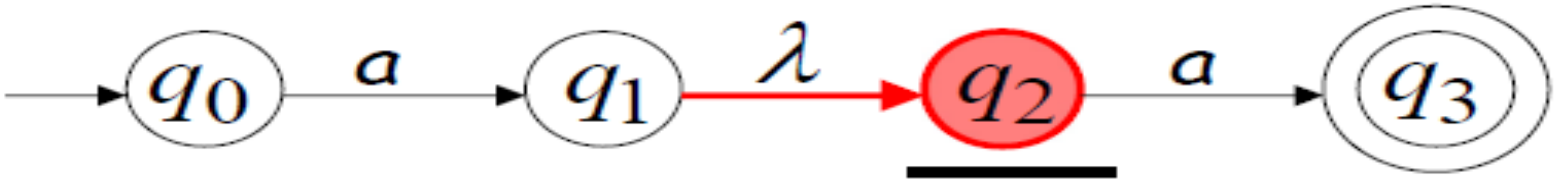
NFA (with λ–transition)



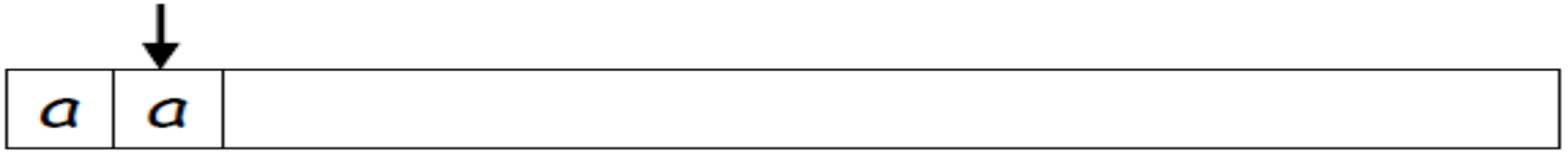


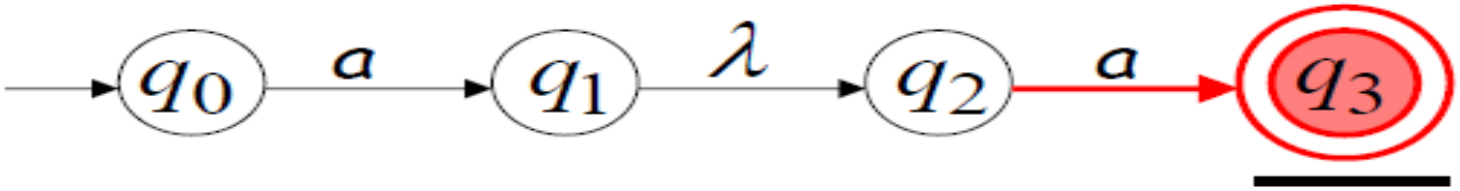
NFA (with λ–transition)





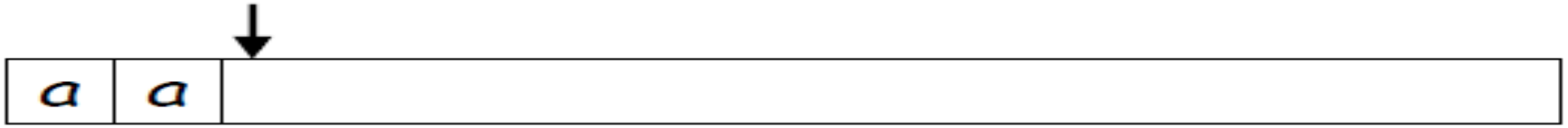
NFA (with λ–transition)

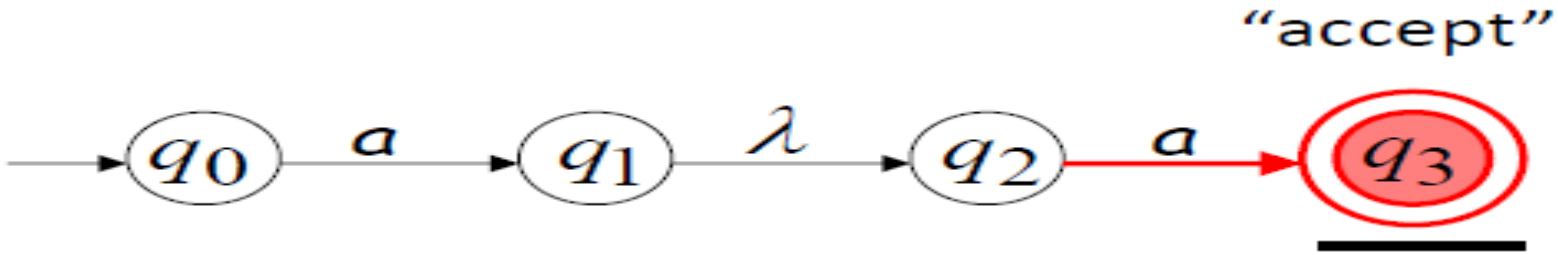




NFA (with λ–transition)

- All input are consumed

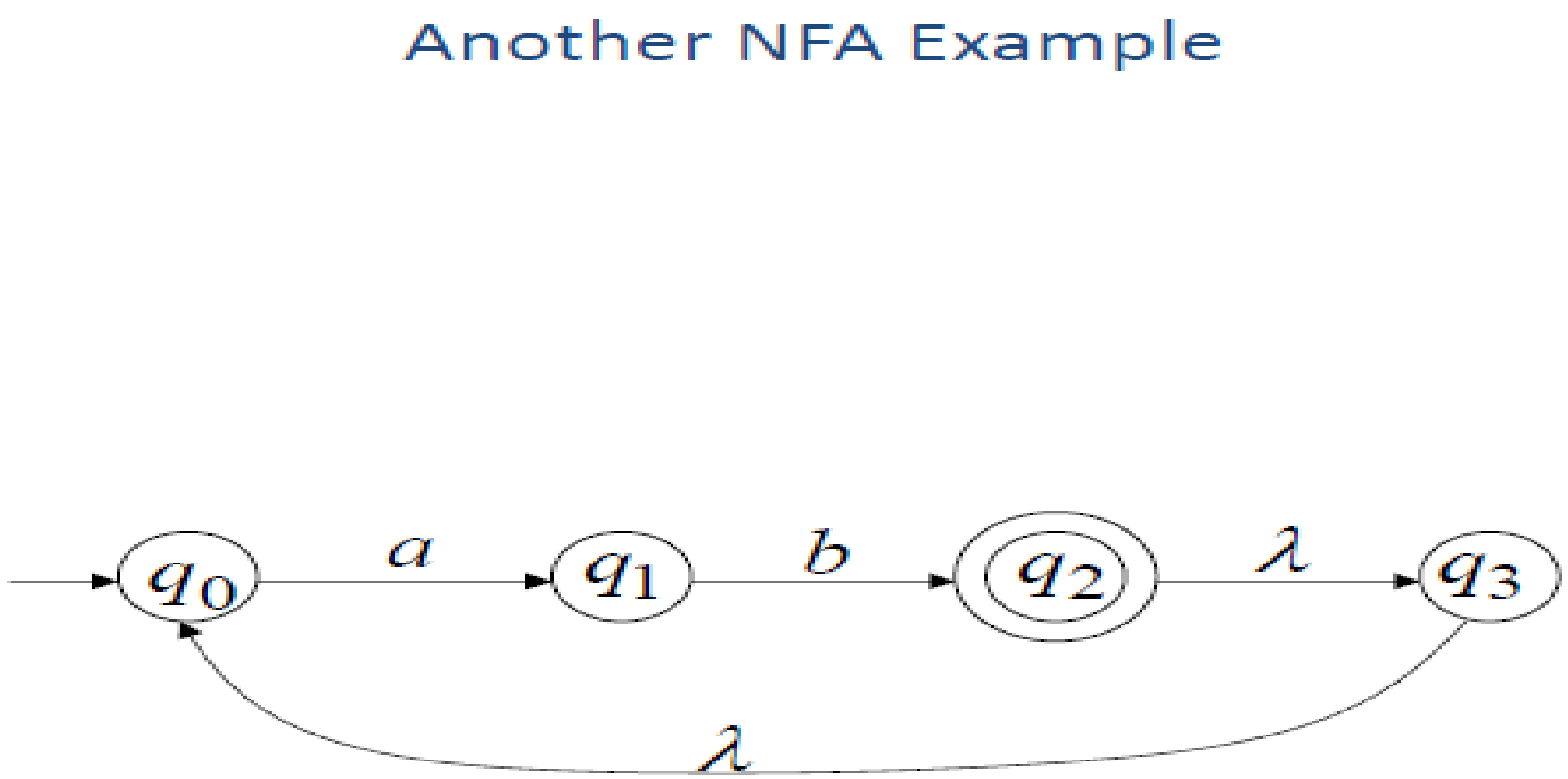




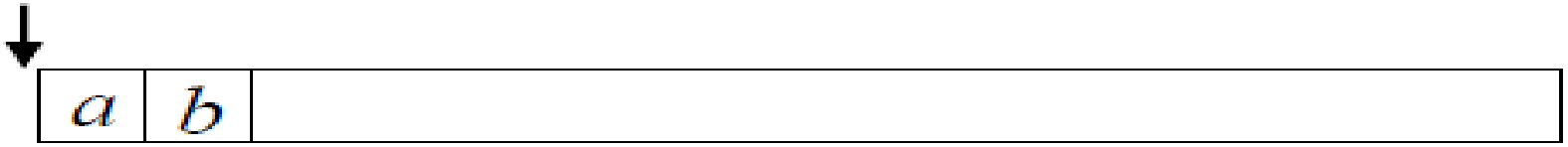
String aa is accepted

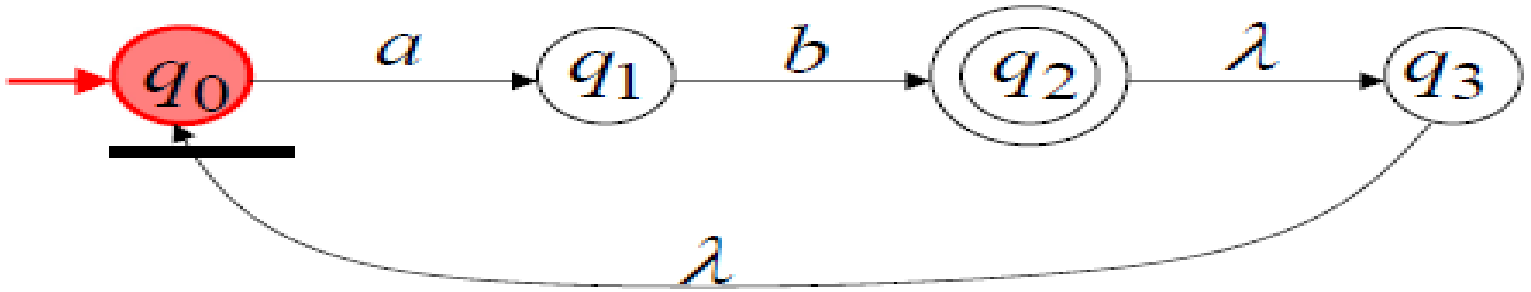
41

Another NFA Example

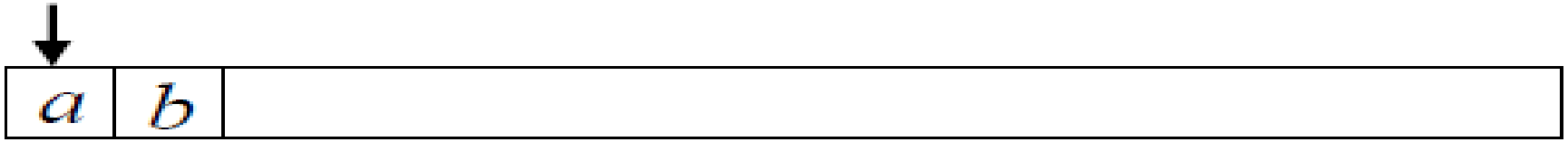


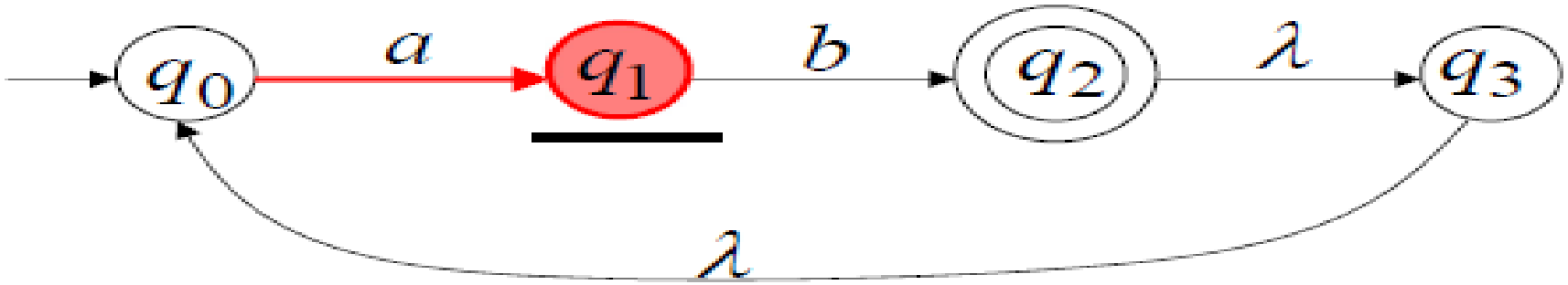
NFA Example



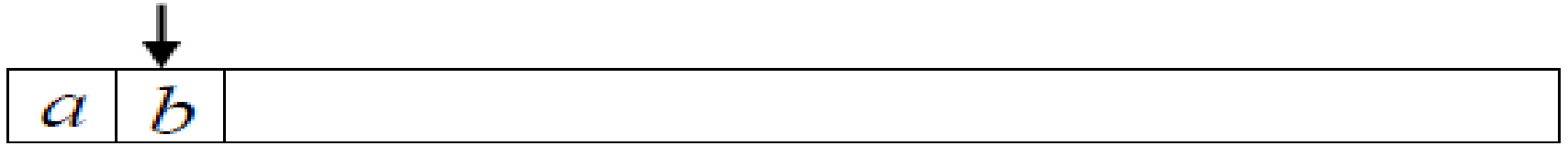


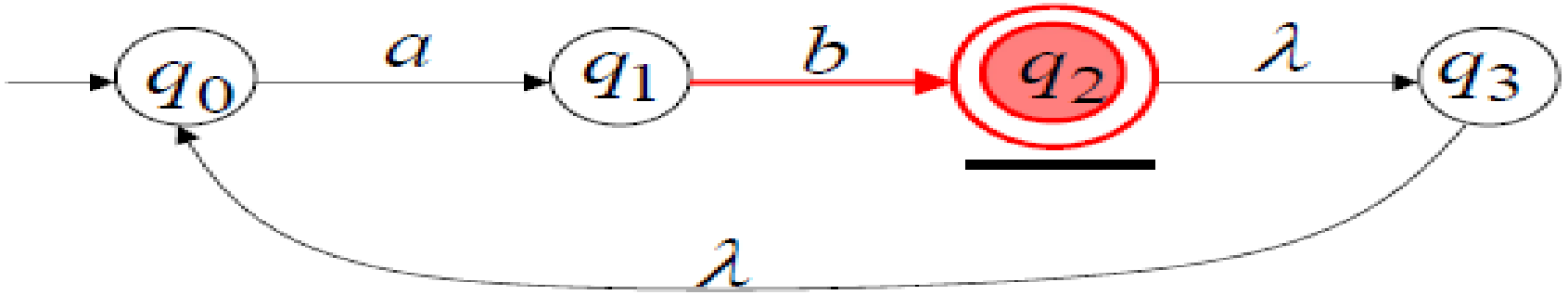
NFA Example



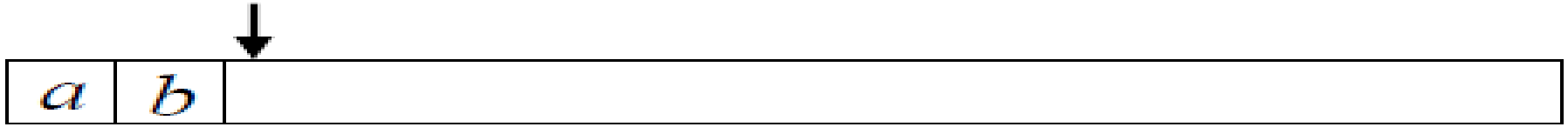


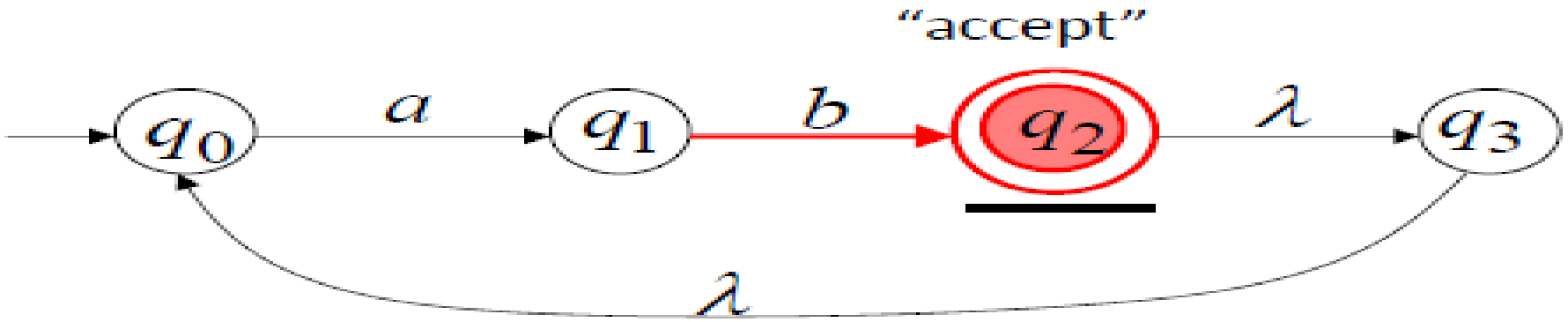
NFA Example



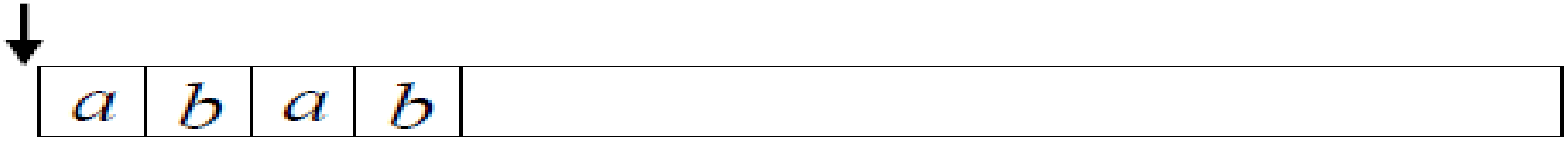


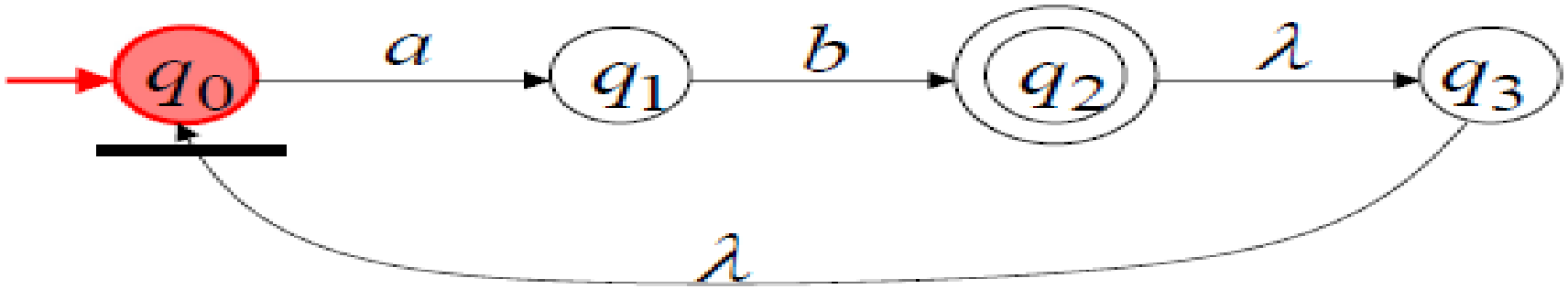
NFA Example



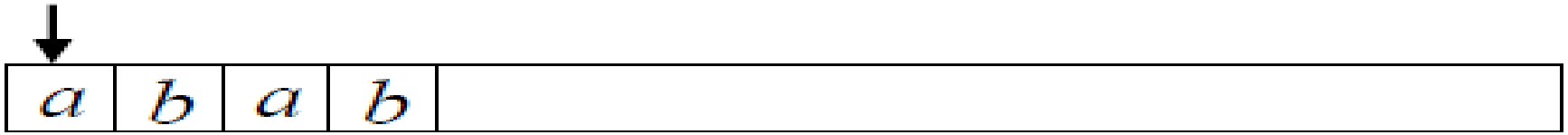


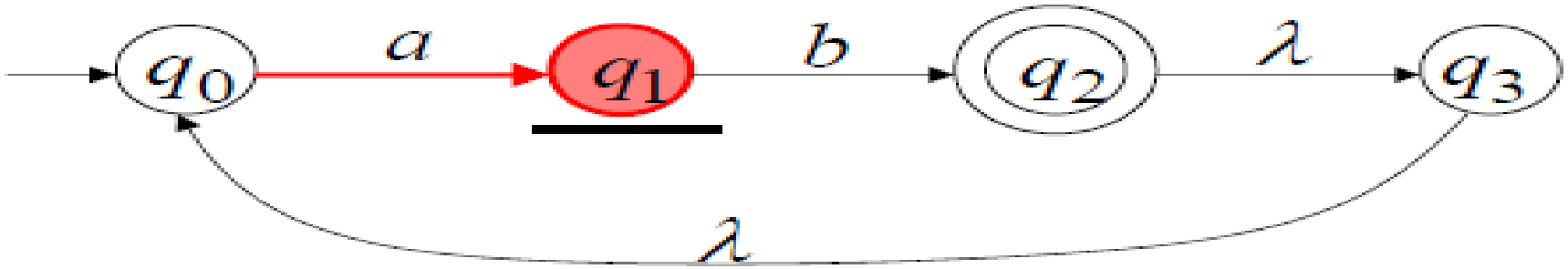
NFA Example (abab string)



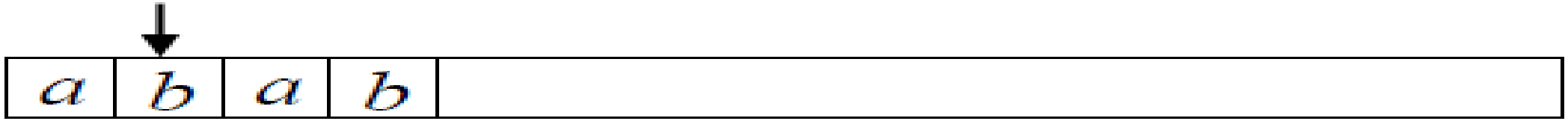


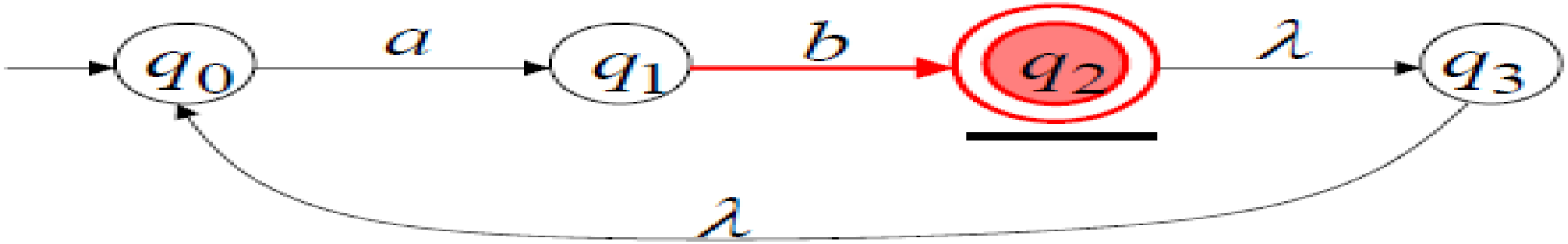
NFA Example (abab string)



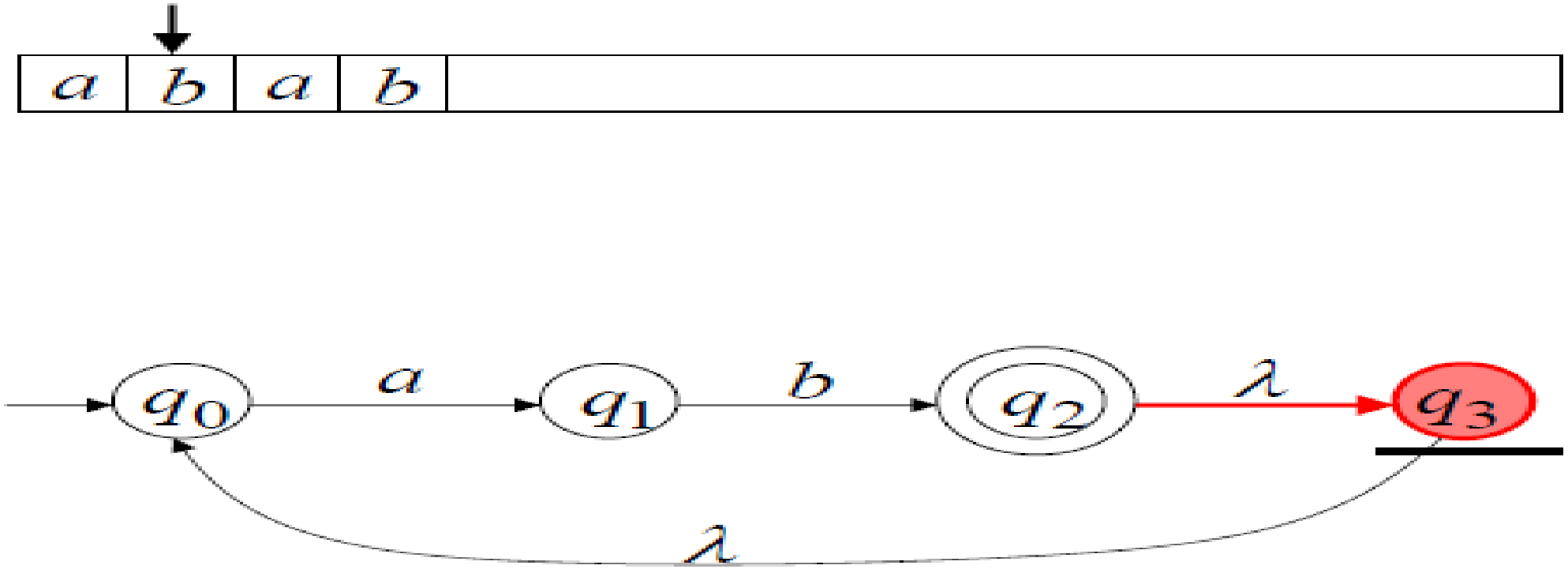


NFA Example (abab string)

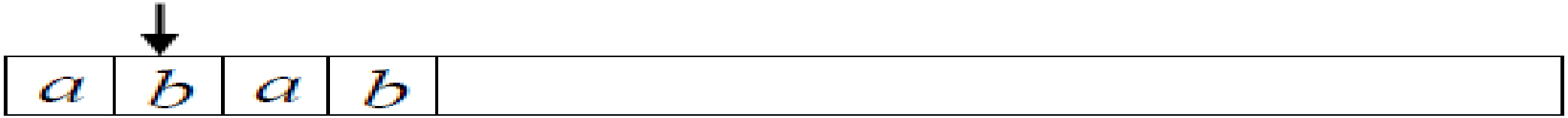


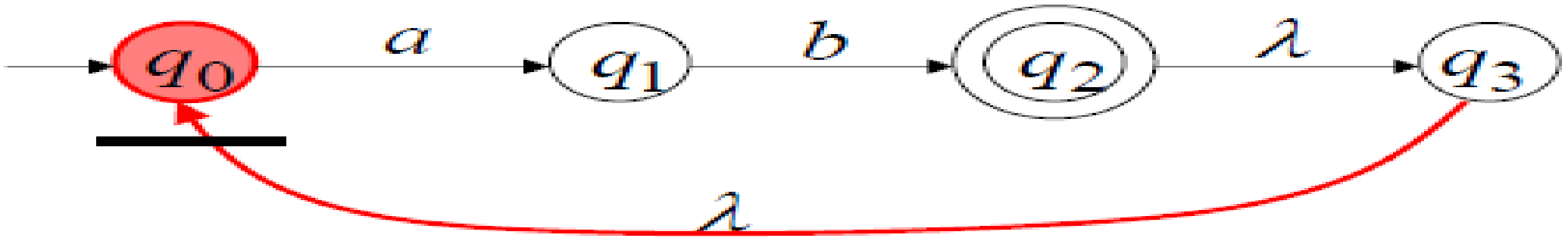


NFA Example (abab string)

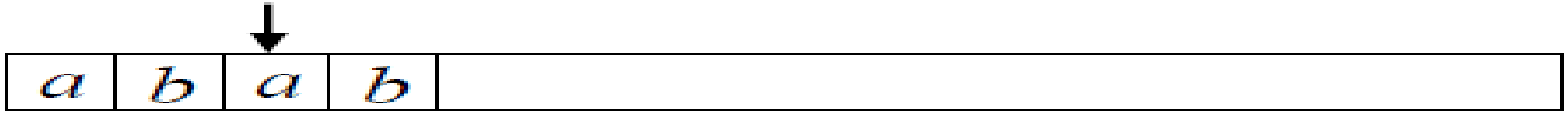


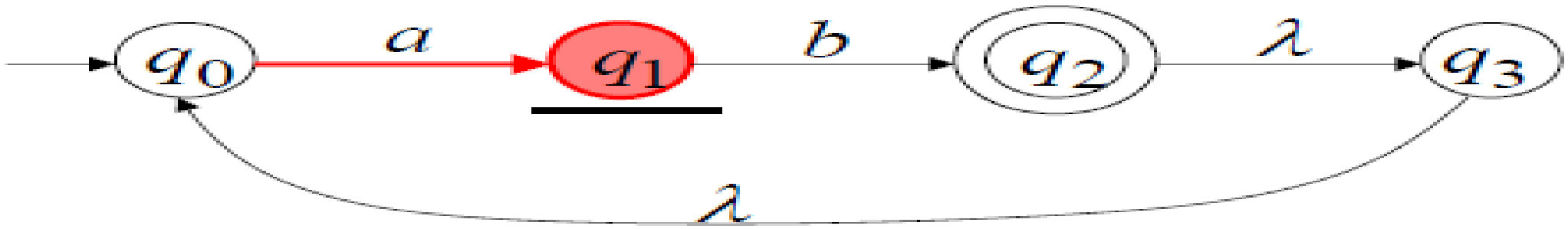
NFA Example (abab string)



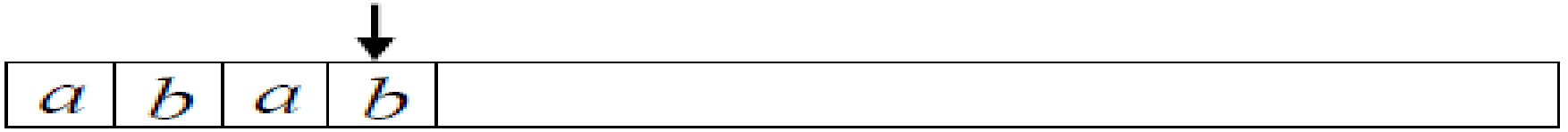


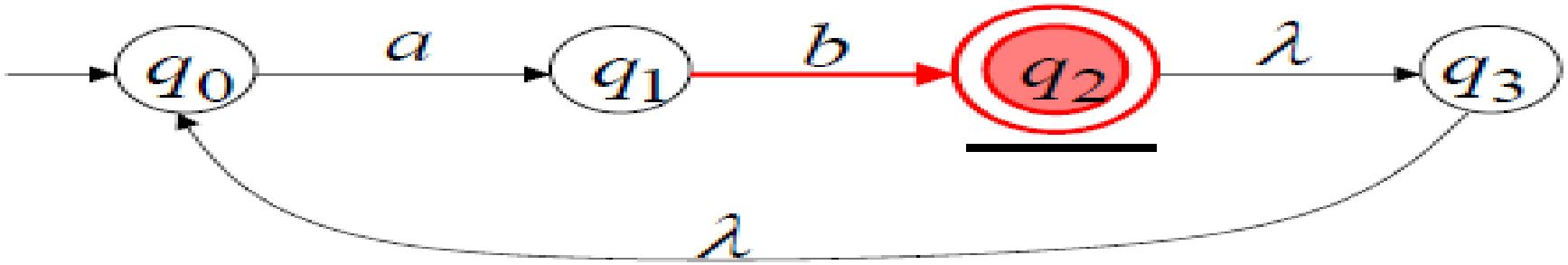
NFA Example (abab string)



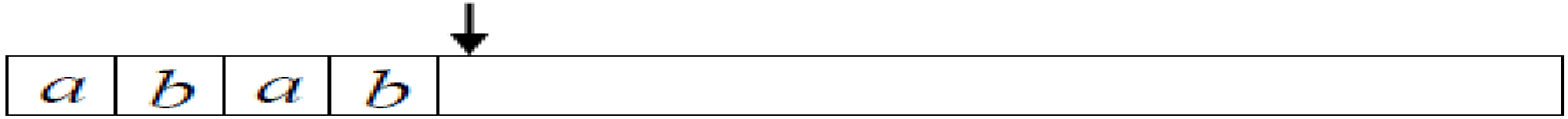


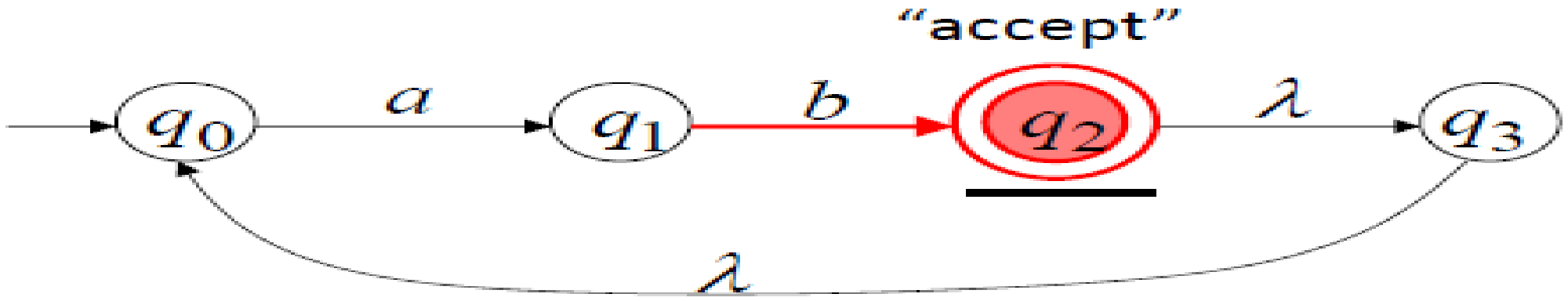
NFA Example (abab string)





NFA Example (abab string)



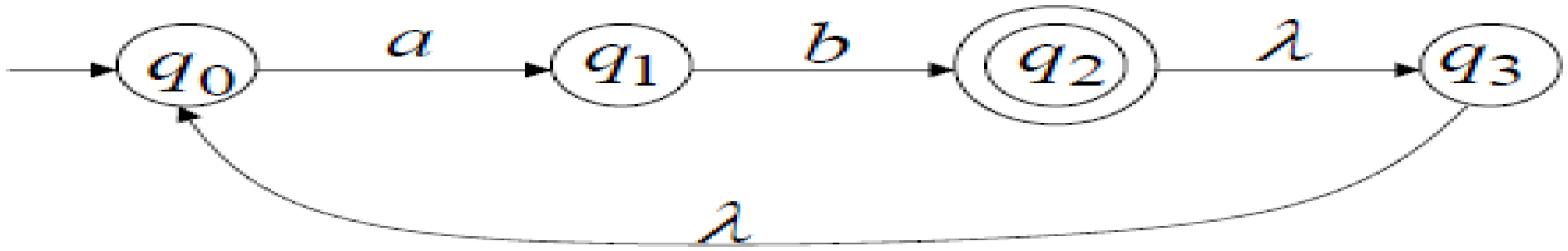


Language accepted

Language accepted

$$L = \{ab,\ abab,\ ababab,\ \ldots\}$$

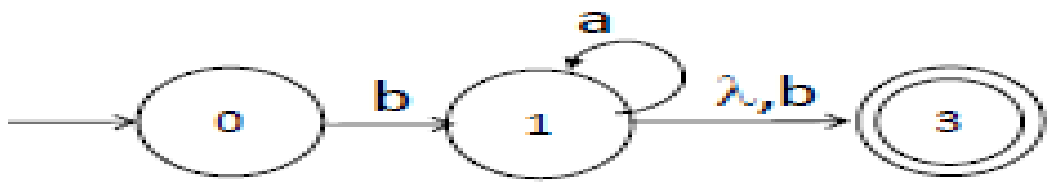
$$= \{ab\}^{+}$$



Example

Tracing NFA (λ–transition)

- Input b, ba, bb, bbb – accept or reject?



Input b

q0 —b→ q1 reject

q0 —λ→ q3 accept

b is accepted

Designing NFA(λ–transition)

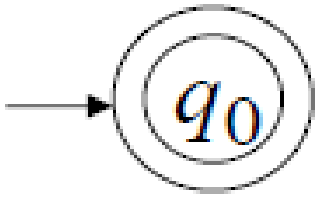
- Simple automata

M_1



$L(M_1) = \{\}$

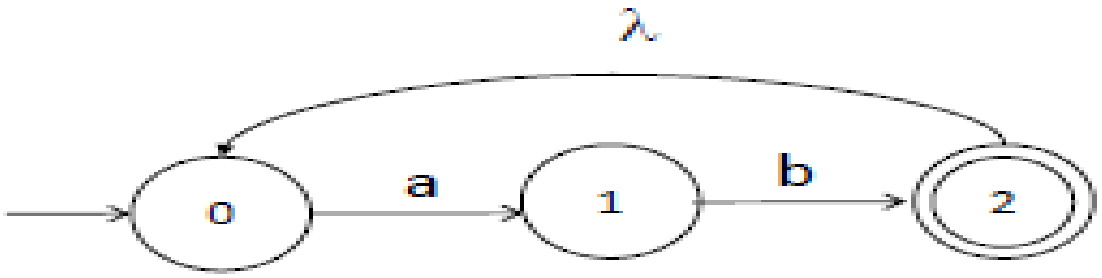
M_2



$L(M_2) = \{\lambda\}$

Design NFA (λ–transition)

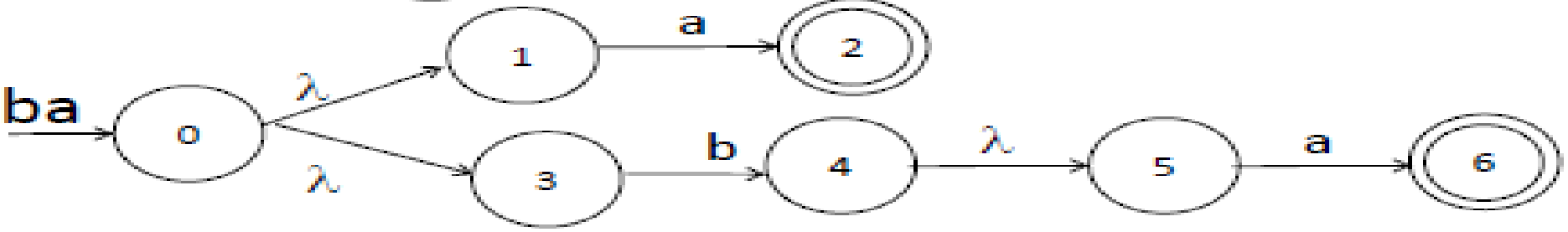
$(ab)^+$



ba



a U ba



Design NFA (λ–transition)

(ab)*ba

